CMPE 344
Microprocessor Lab
Spring 2024

Group 5 - LIDAR Sensor Report

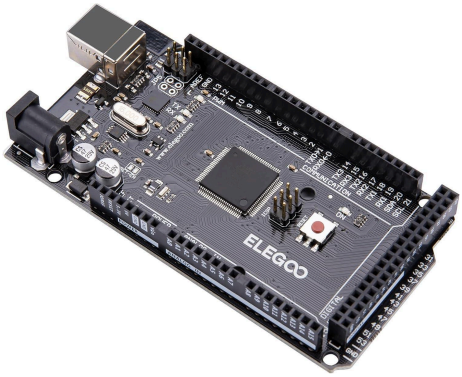
Document Prepared by: Chris Ruiz-Guerra, Nabil Alhamal, Nahom Solomon

Project Description **2**
Theory of Operation 2
BOM **3**
Block diagram **4**
Circuit Description **5**
Code Description **6**
Learning Experience **7**
Hardware Issues 7
Software Issues 7
Future Directions 8
Conclusion **9**

Project Description

Using a microcontroller development board, a Light Detection and Ranging (LIDAR) sensor will measure distance, and output results to a display module. The chosen development board is an ATmega 2560 since it has multiple Serial ports and more Digital Pins than more commonly available ATmega328p. The completed project will be housed and mobile, being powered by a 9V battery.

Theory of Operation



The LIDAR sensor module communicates with the microcontroller using the Inter-Integrated Circuit (I2C) communication standard. The microcontroller (ATmega 2560) is part of a development board similar to an Arduino Atmega 2560. I2C communicates serially by synchronizing clocks on an SCL pin, and data is passed between the devices over the SDA pin.

The LIDAR sensor sends a Light pulse, and measures the start time and return time, since the speed of light is a known constant and can be used in the calculation. The sensor's operating range according to the module manufacturer's manual indicates 0.2 - 8m with a Margin of Error within 6cm (0.2 to 3m) and 2% (3m to 8m). (Benewake (Beijing) Co, Ltd)

The collected data from the LIDAR sensor is stored on the microcontroller, and output as a string to an LCD screen. The communication with the LCD screen is in parallel over 4 bits. The screen has two rows, each row being able to display sixteen characters (Hitachi, Ltd).

The complication of sending data serially or in parallel is abstracted away with publicly available Arduino Libraries, specifically Wire.h (for I2C), LiquidCrystal.h (for the LCD), and TFLI2C.h (for the LIDAR sensor). A few objects are declared, and written code only has to invoke the class methods.

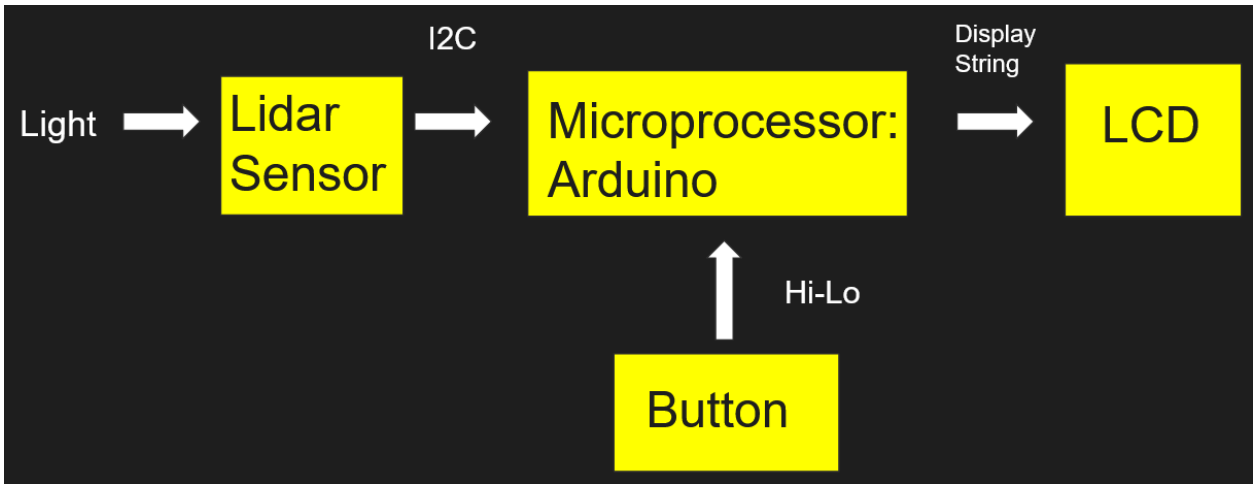
Finally, a pushbutton is used to read high, but if read low, would collect data from the LIDAR Sensor and output the data to the LCD Screen.

BOM

On-Hand ?	**Item Description**	**Lead time (weeks)**	**Qty**	**Unit Price**	**Total Price**
y	ELEGOO MEGA R3 Board ATmega 2560	1	1	$20.99	$20.99
	Benewake TF-Luna LiDAR Module	1	1	$26.59	$26.59
	400 Points Solderless Breadboard	1	1	$6.69	$6.69
y	120pcs Breadboard Jumper Wires	1	1	$6.98	$6.98
y	1602 Serial LCD Module Display	1	1	$9.99	$9.99
y	9V Battery	1	1	$3.00	$3.00
	9V Battery Holder	0	1	$2.25	$2.25
	Ammo Box (housing)	0	1	$5.99	$5.99
	Push button	0	1	$1.33	$1.33
y	Crimp Pins Housing (6 pin male header)	0	1	$0.02	$0.02
y	Crimp Pins (male pins)	0	6	$0.02	$0.12
	Stand-offs	0	12	$0.03	$0.34
	M2 screws	0	12	$0.03	$0.34
				subtotal	$43.54
				tax	$4.68
				total	$48.22

Block diagram

The flow of the project was to have the Lidar sensor send light to an object which is reflected back to the Lidar sensor, this is how it detects distance. The Lidar sensor communicates with the Arduino microcontroller with I2C communication. The code is running on the microcontroller based on the input received from the button. When the button is pressed the circuit is grounded and reads as Low, when not pressed the circuit is open and reads as high. When the circuit is Low is when the microcontroller runs the code. The data is then communicated to display on the LCD screen



Circuit Description

The Arduino is supplied with 9V from an external battery, and then feeds 5V to the rest of the circuit. The 9V battery pack is not shown in the circuit diagram.

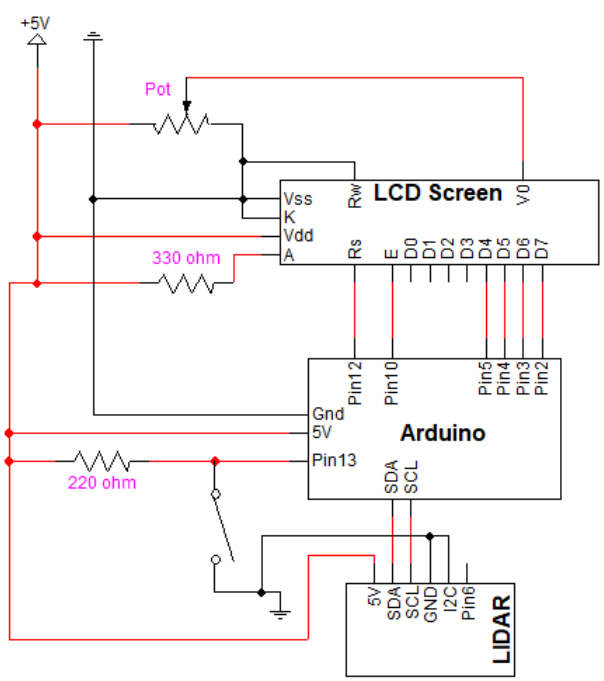
The LCD screen has pin V0 as a voltage input to adjust the screen contrast using a potentiometer. Pin A is the input voltage for the LCD backlight, and pins K, Vss, and RW are grounded. RW is grounded, indicating that the device is in Write mode. Pin E starts the write, RS selects which registers to write to. Finally, D4 to D7 is the Data bus, acting in 4-bit mode. If we were using 8-bit mode, pins D0 to D3 would be used. At this time, they are not. (Hitachi, Ltd)

Signal	No. of Lines	I/O	Device Interfaced with	Function
RS	1	I	MPU	Selects registers. 0: Instruction register (for write) Busy flag: address counter (for read) 1: Data register (for write and read)
R/$\overline{W}$	1	I	MPU	Selects read or write. 0: Write 1: Read
E	1	I	MPU	Starts data read/write.
DB4 to DB7	4	I/O	MPU	Four high order bidirectional tristate data bus pins. Used for data transfer and receive between the MPU and the HD44780U. DB7 can be used as a busy flag.
DB0 to DB3	4	I/O	MPU	Four low order bidirectional tristate data bus pins. Used for data transfer and receive between the MPU and the HD44780U. These pins are not used during 4-bit operation.

The LIDAR sensor has six pins, and their functions depend on whether we are using I2C or UART mode. Pin 1 is connected to our 5V supply from the Arduino, Pin 2 connects to the SDA pin, Pin 3 to the SCL pin, Pin 4 and Pin 5 to Ground, and Pin 6 is unused since we are not communicating to multiple devices (it's used as a multiplexed output). (Benewake (Beijing) Co, Ltd)

No.	Function	Description
1	+5V	Power supply
2	RXD/SDA	Receiving/Data
3	TXD/SCL	Transmitting/Clock
4	GND	Ground
5	Configuration Input	Ground: I2C mode /3.3V: Serial port Communications mode
6	Multiplexing output	Default: on/off mode output I2C mode: Data availability signal on but not switching value mode

We also have current limiting resistors in the circuit, specifically to Pin A on the LCD screen, and another to Pin 13 of the Arduino. Pin 13 connects to our normally open pushbutton, which grounds Pin 13 when pushed.



Code Description

The inclusion of Arduino libraries Wire.h, LiquidCrystal.h, and TFLI2C.h simplified the programming to instantiating class objects, and using class methods for most functionality we wanted to implement.

We wrote the program using the Arduino IDE. This also allowed us to upload our compiled program to the Arduino,making this a preferred reason to use an Arduino microcontroller.

We instantiated a LiquidCrystal object called lcd using an overloaded constructor with the pin numbers for RS, E, and D4 to D7. We also instantiated a TFLI2C object called tfII2C. We also declared integer variables tfDist to store distance in centimeters, tfAddr to store the default I2C address from the TFLI2C library, and Button was set to 13.

In the setup block of the code, we initialized the Wire library, initialized the Serial baud rate to the Serial Monitor (for testing purposes), and initialized the interface of the LCD screen - defining the rows to be 2 and the columns 16, and setting Button as an input.

In the loop block, we placed our running code. The loop has two conditional branches using if statements. The outer branch reads the Button and tests to see if it is low (ie, the button is depressed and pin 13 is set to ground). The code then repeats with a delay of 100ms at the top of the block.

The inner branch is also an if statement that uses the TFLI2C method getData() , which takes two parameters, tfDist and tfAddr. tfDist is passed by reference, so the parameter is in In-Out mode. The method returns a boolean value. The code to this method makes calls to Wire.h methods to get data from the LIDAR sensor. If this condition is true, we will carry out methods to display to the LCD screen. We use methods clear(), setCursor(int y, int x), and print(string s) to clear the screen, set the position of the cursor by column and row, and print s.

Using these methods reduced the code to just under 40 lines.

Learning Experience

During this process, we ran into several problems that were not anticipated. We understood that there would be difficulties along the way, so planned a development cycle to increment in small pieces. We began with simply testing to see if we could communicate with the serial monitor with Arduino. We then worked on displaying the same text to the LCD screen. Once that was successful, we worked on the reading data from the LIDAR sensor. Once this was working, we implemented both components and added a pushbutton module. The final step was to place the assembly into a casing for safety, and ease of transportation, and for demonstration purposes.

Hardware Issues

We initially had issues with our pinouts between the LCD screen and Arduino. These were resolved pretty quickly after referencing the HDD44780 Datasheet, which is the main controller for the LCD screen. We also had trouble implementing the pushbutton to read to pin 13. After drawing out a schematic and getting some advice from Dr. Sumarsono, we found that we didn't have proper resistance from the voltage source to the pin. An adjustment to the schematic solved the problem.

Software Issues

The first issue with using the libraries came up when trying to display text to the LCD screen. We learned that we needed to declare the size of the display (how many rows, and how many columns of characters). After referencing the library documentation, this process became simple.

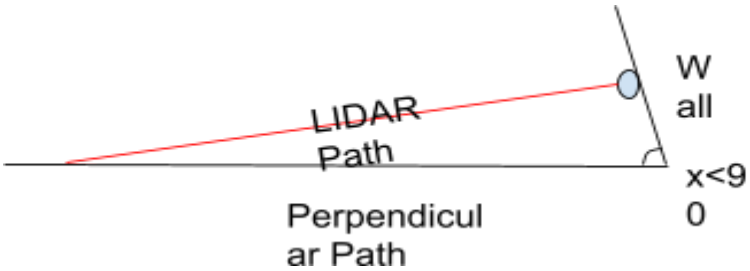
The second issue diverged our group into working on a solution separately. We worked on implementing data collection from the LIDAR sensor using UART or I2C. After many failed attempts following the LIDAR documentation, we learned that using the Serial library with the serial ports on the Arduino is not well supported. We found a Serial library, but no library that implemented Serial methods to give us simple methods to extract data from the device. We attempted to build these functions but ran into several issues, the major one being that we received no data. The documentation stated that there are 9 bytes of data, and we need to verify the bytes are correct according to the image shown from the documentation, that is, the first two bytes should be 0x59, and the last byte should be a checksum. This problem was intractable with the time available, and the second group found a device library publicly available to use.

Byte	0	1	2	3	4	5	6	7	8
Description	0x59	0x59	Dist_L	Dist_H	Amp_L	Amp_H	Temp_L	Temp_H	Check_sum

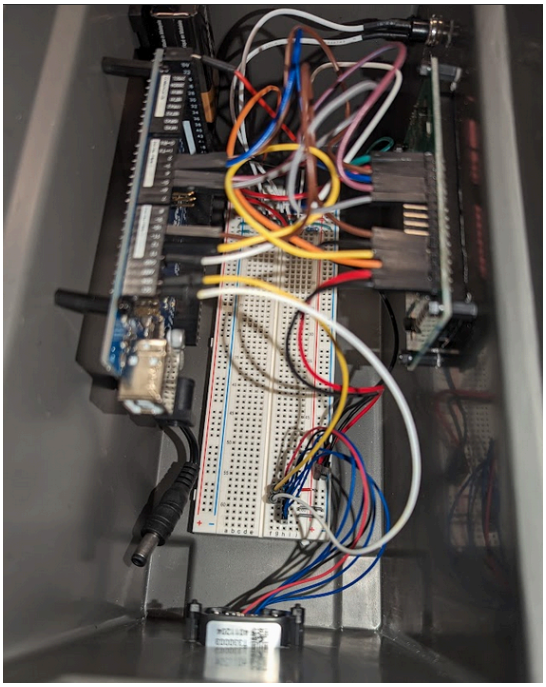
(Benewake (Beijing) Co, Ltd)

Future Directions

During testing, we found some things that could be improved on this prototype. The first thing we found after assembling the device into the housing, we didn't know if we were accurately pointing at the object we wanted to measure distance to. We considered adding a laser pointer earlier in the development phase but opted not to because we improperly assumed that we would be pointing at an object directly so we didn't need a visual indicator verifying this. During testing, we found that our read values after a certain distance indicated no changes. We realized that the housing wall in which the LIDAR sensor was attached to was slightly tapered, pointing the sensor slightly downward. We learned that some kind of calibration was likely a needed feature in a device like this in order to trust the measurements.

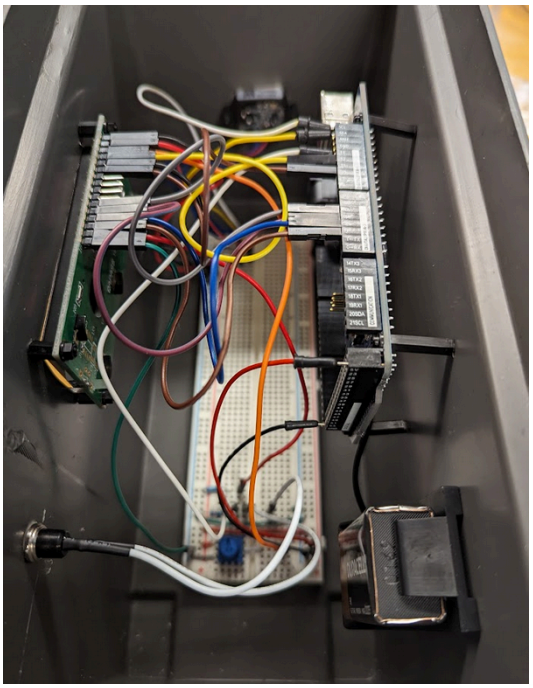


We also found that not having a power switch required us to open the box and unplug the battery pack from the rest of the circuit. To turn it on, we have to reconnect it to power the device. A power switch on the outside would have helped solve this problem.



Conclusion

We built a LIDAR sensor using a microcontroller similar to an Arduino ATmega2560. We implement an output to an LCD Screen for easy reading by the user. We also implemented a pushbutton for the user to select when to take a measurement and display the output. We attached all components to a plastic, covered housing using stand-offs. Holes were cut into the housing to allow for the LCD screen to be visible from outside the housing, and one for the LIDAR sensor at the front to be able to take a measurement. The completed project was mobile and powered by a 9V battery.



Code

```
#include <Wire.h>        // Instantiate the Wire library. This is used for I2C communication
#include <TFLI2C.h>      // TFLuna-I2C Library v.0.1.1. Requires Wire.h library
#include <LiquidCrystal.h> //Library for LCD

LiquidCrystal lcd(12, 10, 5, 4, 3, 2);  //pins used for the LCD screen. 12 and 10 are the RS and EN pins, and 5,4,3,2 are connected to D4, D5, D6, and D7 respectviely.

TFLI2C tfl12C; //class TFLI2C
int tfDist;    // distance in centimeters
int tfAddr = TFL_DEF_ADR;  // Use this default I2C address or

const int Button= 13;  // sets the digital pin 13 as input

void setup()
{
    Serial.begin( 115200);  // Initalize serial port
    Wire.begin();           // Initalize Wire library
    lcd.begin(16, 2);   // putting in the diaentions of the LCD
    pinMode(Button, INPUT); // setup pin 13 as input to read the button
}

void loop()  // Forever loop
{
    delay(100); // delay for our eyes to be able to see the LCD properly
    if (digitalRead(Button) == LOW){ //  if the digital pin voltage reading is at ZERO / Ground, The LIDAR reading will continue
      if( tfl12C.getData( tfDist, tfAddr)) // Return Bool if data is read, tfDist is passed by reference, tfAddress is passed by value
      {
          lcd.clear();               // clear LCD screen
          lcd.setCursor(2,0);        // sets cursor to row 0 column 2
          lcd.print("Dist: ");       // print "Dist: " on LCD
          lcd.print(tfDist);          // print the data...
          lcd.print("cm ");          // print "cm " on LCD
      }
    }
      else{
      tfl12C.printStatus();           // Debugging to serial monitor: Ready or Print Error
      delay(100);                     //delay for our eyes to be able to see the serial monitor properly
  }
}
```

Works Cited

Amazon.com. “Benewake TF-Luna LiDAR Module Range Finder Sensor Single-Point Micro Ranging Module for Arduino Pixhawk 5V UART IIC Interface.” *Amazon*, Unknown, https://www.amazon.com/Benewake-TF-Luna-Single-Point-Ranging-Interface/dp/B086MJQSLR/ref=sr_1_1_sspa?crid=1DA83PPRIY041&keywords=lidar+sensor&qid=1706830956&sprefix=lidar+sensor%2Caps%2C113&sr=8-1-spons&sp_csd=d2lkZ2V0TmFtZT1zcF9hdGY&psc=1. Accessed 1 May 2024.

Amazon.com. “ELEGOO MEGA R3 Board ATmega 2560 + USB Cable Compatible with Arduino IDE Projects RoHS Compliant.” *Amazon.com*, Unknown, https://www.amazon.com/ELEGOO-ATmega2560-ATMEGA16U2-Arduino-Compliant/dp/B01H4ZDYCE?sp_csd=d2lkZ2V0TmFtZT1zcF9hdGY&th=1. Accessed 1 May 2024.

Amazon.com. “SunFounder IIC I2C TWI 1602 Serial LCD Module Display Compatible with Arduino R3 Mega 2560 16x2.” *Amazon.com*, Unknown, https://www.amazon.com/SunFounder-Serial-Module-Display-Arduino/dp/B019K5X53O/ref=sr_1_1_sspa?crid=1N30YS7HYJG9Y&keywords=lcd%2Bdisplay%2B2x%2B16&qid=1706832192&sprefix=lcd%2Bdisplay%2B2x%2B16%2Caps%2C124&sr=8-1-spons&sp_csd=d2lkZ2V0TmFtZT1zcF9hdGY&th=1. Accessed 1 May 2024.

Arduino. “Libraries.” *Arduino*, Unknown, https://www.arduino.cc/reference/en/libraries/. Accessed 1 May 2024.

Benewake (Beijing) Co, Ltd. “Product Manual of TF-Luna.” *Cloud Object Storage - Amazon S3*, https://s3-us-west-2.amazonaws.com/files.seeedstudio.com/products/101990656/res/SJ-PM-TF-Luna+A01+Product+Manual.pdf. Accessed 1 May 2024.

Hitachi, Ltd. “HD44780U (LCD-II), (Dot Matrix Liquid Crystal Display Controller/Driver).” *SparkFun Electronics*, Hitachi, Ltd, 1998, https://www.sparkfun.com/datasheets/LCD/HD44780.pdf. Accessed 1 May 2024.

Reyerson, Bud. “TFLuna-I2C Source Code.” *GitHub.com*, 9 November 2017, https://github.com/budryerson/TFLuna-I2C/tree/master/src. Accessed 1 May 2024.